



LIONSDEV

Curso de Programação

MÓDULO 5

Modularização de aplicações Node.js



Professor Nicolas Cardoso Motta

Revisando o Módulo 4



Commit

`git add` escolhe o que entra; `git commit -m` salva uma foto do projeto.



Remoto

`git push` envia, `git pull` traz e `git clone` baixa o repositório inteiro.



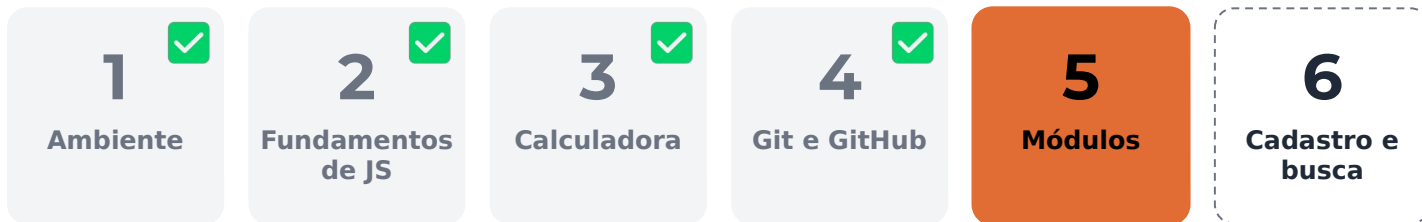
Branches e PR

Cada tarefa numa branch; o Pull Request revisa antes do merge na `main`.

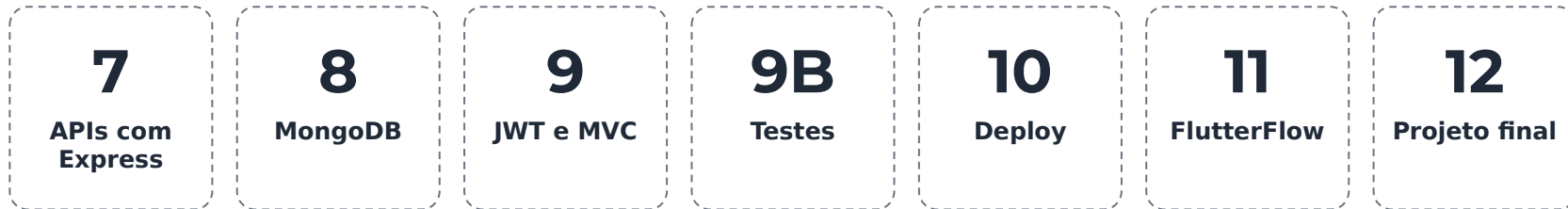
Pergunta para abrir: por que a pasta `node_modules` vai no `.gitignore`?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



Você está aqui: **Módulo 5**, Modularização de aplicações Node.js



Objetivo

- Entender o que é um módulo e por que dividir o código em arquivos.
- Usar os módulos nativos do Node.js.
- Diferenciar CommonJS e ES Modules.
- Exportar e importar com export default e export nomeado.
- Refatorar a calculadora do Módulo 3 em módulos.

No Módulo 3, a calculadora inteira ficou num arquivo só. Funciona... mas imagine um projeto com milhares de linhas.

O problema do código monolítico

Difícil de ler

Centenas ou milhares de linhas num único arquivo.

Difícil achar bugs

Tudo misturado: menu, regras e contas.

Difícil em equipe

Todo mundo mexendo no mesmo arquivo gera conflitos no Git.

Nada se reaproveita

Para usar uma função em outro projeto, só copiando e colando.

Monolítico = tudo num lugar só. A solução é dividir o código em módulos.



O que é um módulo?

Um módulo é um arquivo



Um arquivo JavaScript

Cada módulo é um arquivo que guarda um pedaço do código, com uma responsabilidade clara.



Escopo próprio

O que está dentro do arquivo só é visível para fora se for exportado.



Exporta e importa

Um arquivo disponibiliza o código (export) e outro traz para usar (import).

Por que modularizar?



Reutilização

Escreva uma vez e use em vários lugares, sem duplicar código.



Manutenção

Arquivos pequenos: é fácil achar e corrigir erros ou adicionar funcionalidades.



Organização

Cada módulo cuida de uma responsabilidade. O projeto fica fácil de ler.



Colaboração

Cada pessoa trabalha num módulo diferente, ao mesmo tempo.



Módulos Nativos

Ferramentas que já vêm com o Node.js

Módulos nativos (core modules)

O Node.js já vem com várias ferramentas "de fábrica". Não precisa instalar nada: basta importar pelo nome.



fs

Lê e cria arquivos no computador (você viu nas ferramentas do Módulo 2).



path

Monta caminhos de pastas e arquivos do jeito certo.



http

Cria servidores web. O Express do Módulo 7 é construído em cima dele.

CommonJS: `const fs = require("fs")` · **ES Modules:** `import fs from "node:fs"`



CommonJS x ES Modules

Dois sistemas de módulos



CommonJS (CJS)

Criado pelo Node.js quando o JavaScript ainda não tinha módulos oficiais.

Usa `require` e `module.exports`.



ES Modules (ESM)

O padrão oficial do JavaScript, usado também nos navegadores e no React.

Usa `import` e `export`.

O Node.js suporta os dois. Como desenvolvedores, precisamos saber ler e escrever ambos.

Exportando e importando

CommonJS

```
// mensagem.js
const mensagem = 'Olá, Lions!'
module.exports = mensagem

// app.js
const mensagem = require('./mensagem')
console.log(mensagem)
```

ES Modules

```
// mensagem.js
const mensagem = 'Olá, Lions!'
export default mensagem

// app.js
import mensagem from './mensagem.js'
console.log(mensagem)
```

Mesma ideia, sintaxe diferente. No ES Modules, o caminho do arquivo leva a extensão .js.

Ativando o ES Modules

```
{  
  "name": "calculadora",  
  "version": "1.0.0",  
  "main": "calculadora.js",  
  "type": "module",  
  "dependencies": {  
    "prompt-sync": "^4.2.0"  
  }  
}
```

"type": "module" → avisa o Node.js que o projeto usa **import** e **export**.

padrão → sem essa linha, o Node.js trata todo arquivo **.js** como CommonJS.

Módulo 2 → lembra do aviso no terminal e do **require** que para de funcionar? É esta linha.



Exportação Padrão x Nomeada

Exportação padrão (default)

```
// somar.js
function somar(a, b) {
  return a + b
}
export default somar

// app.js
import somar from './somar.js'
import qualquerNome from './somar.js' // também funciona
```

export default → a coisa principal do arquivo. Só pode haver uma por arquivo.

import → sem chaves, e o nome é livre: você escolhe como chamar.

quando usar → quando o arquivo tem uma única responsabilidade, como **somar.js**.

Exportação nomeada (named)

```
// operacoes.js
export function somar(a, b) {
  return a + b
}
export function subtrair(a, b) {
  return a - b
}

// app.js
import { somar, subtrair } from './operacoes.js'
```

export → na frente de cada coisa que o arquivo disponibiliza. Pode haver várias.

{ } → no import, os nomes vêm entre chaves e precisam ser exatamente iguais aos do export.

quando usar → quando o arquivo reúne várias funções relacionadas.



Cuidado: o ./ e o .js são obrigatórios

No ES Modules, o caminho de um arquivo seu precisa do ./ e da extensão .js.

- Sem o ./, o Node.js procura um pacote na `node_modules` e dá `ERR_MODULE_NOT_FOUND`.
- Sem o .js, também dá erro: diferente do `require`, o `import` não completa a extensão.
- No Linux, maiúsculas contam: `./Somar.js` e `./somar.js` são arquivos diferentes.

```
import somar from 'somar.js'    // procura um pacote na node_modules
import somar from './somar'     // falta a extensão
import somar from './somar.js'  // correto
```



Cuidado: chaves e nomes no import

Import nomeado usa **chaves** e o **mesmo nome** do export. O default não usa chaves.

- Trocar um pelo outro é o erro mais comum de quem está começando com módulos.
- O Node.js avisa: "does not provide an export named..." ou "does not provide an export named default".

```
// operacoes.js tem: export function somar(a, b) { ... }  
import { somar } from './operacoes.js'    // correto  
import somar from './operacoes.js'       // erro: não existe export default  
import { soma } from './operacoes.js'    // erro: o nome é somar
```

Cada módulo tem o seu escopo

```
// contador.js
let total = 0
export function incrementar() {
  total++
  return total
}

// app.js
import { incrementar } from './contador.js'
console.log(incrementar()) // 1
console.log(total)         // ReferenceError: total is not defined
```

O que não é exportado fica privado do módulo. E o prompt também: cada arquivo que usa prompt precisa importar o prompt-sync.



Cuidado: caminhos entre pastas

O caminho do import é **relativo ao arquivo que está importando**, não à pasta onde você rodou o node.

- `./` é a pasta do próprio arquivo; `../` sobe uma pasta.
- De `src/routes/livros.js` para `src/models/livro.js`:
`../models/livro.js`.

```
src/  
├── routes/livros.js    import Livro from "../models/livro.js"  
└── models/livro.js
```



Modularizando a Calculadora

Um arquivo por operação

```
calculadora/  
├── package.json  
├── calculadora.js      (menu e laço)  
├── somar.js  
├── subtrair.js  
├── multiplicar.js  
├── dividir.js  
└── porcentagem.js
```

calculadora.js → o arquivo principal: importa as operações e cuida só do menu e do laço.

uma operação → cada arquivo exporta uma única função, com **export default**.

extensível → uma operação nova é um arquivo novo e um import. O resto não muda.

Cada operação no seu arquivo

somar.js

```
function somar(num1, num2) {  
  return num1 + num2  
}  
  
export default somar
```

porcentagem.js

```
function porcentagem(valor, percentual) {  
  return (valor * percentual) / 100  
}  
  
export default porcentagem
```

subtrair.js, multiplicar.js e dividir.js seguem o mesmo modelo.

O arquivo principal: calculadora.js

```
import promptSync from 'prompt-sync'
import somar from './somar.js'
import subtrair from './subtrair.js'
import multiplicar from './multiplicar.js'
import dividir from './dividir.js'
import porcentagem from './porcentagem.js'

const prompt = promptSync({ sigint: true })

// ... o mesmo menu, laço e switch do Módulo 3,
// com a mesma validação em cada case
```

**O menu e a validação continuam iguais: só as funções saíram do arquivo.
Separar em módulos não é motivo para tirar a validação da entrada.**



Hora da Prática

Atividade · Calculadora Estatística

Módulos

Desenvolva do zero uma calculadora estatística no terminal, já com a arquitetura de módulos:

1. Inicie o projeto (`npm init -y`), instale o `prompt-sync` e ative o ES Modules (ou use CommonJS).
2. O arquivo principal tem um menu que roda em laço até o usuário escolher sair.
3. Opções: adicionar número, remover número, calcular a média, calcular a mediana e ver a lista.
4. O arquivo principal não faz contas: crie `media.js`, `mediana.js`, `adicionar.js` e `remover.js`.
5. Exporte as funções e importe no arquivo principal, chamando cada uma pela opção do menu.



Dica: converta a entrada com `parseFloat` e confira se a lista não está vazia antes de calcular: média de uma lista vazia divide por zero.



Cuidado: a mediana precisa da lista ordenada

O `sort()` sem parâmetro ordena como **texto**: `[10, 9, 2].sort()` dá `[10, 2, 9]`.

- A mediana é o valor do meio da lista ordenada. Sem ordenar, o resultado sai errado.
- Para números, passe a regra de comparação: `sort((a, b) => a - b)`.
- O `sort` altera o array original. Para não bagunçar a lista do usuário, ordene uma cópia: `numeros.slice().sort(...)`.

```
console.log([10, 9, 2, 1].sort())           // [1, 10, 2, 9] (como texto!)
console.log([10, 9, 2, 1].sort((a, b) => a - b)) // [1, 2, 9, 10]
```

Lista de exercícios



Parte 0 e 1: aquecimento e complete o código
Exports por default e exports nomeados.



Parte 2: ache o bug
Import quebrado e default x nomeado.



Parte 3: prever o resultado
Vai rodar ou dar erro?



Parte 4: desafio
Um mini projeto modular, escrito do zero.

No repositório: `modulo05/lista_de_exercicios/codigo_modulos.md`

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo05](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo05)



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 6, os módulos vão organizar um sistema de cadastro completo.